FINAL PROJECT DOCUMENTATION

Managing Sentry

A POINT-OF-SALE AND INVENTORY MANAGEMENT SYSTEM

> *Custom-built exclusively for Alekos' Corner General Merchandise*

GROUP NUMBER	GROUP MEMBERS	DATE
SWU-009	Aldave, Resty S. Bagahansol, Shan Lester S. Llaguno, Clary Mae F. Rama, John Wilfred Y.	October 2026

Table of Contents

Project Summary....3
Overview....3
Problem Statement....3
Purpose and Objectives....3
Target Users....3
Scope and Limitations....3
System Use Cases....5
PST-01 Process Sales Transaction....5
Use Case Diagram....8
INV-01 Manage Products and Inventory....8
Use Case Description....9
Use Case Diagram....10
Figure 2. Manage Products and Inventory use case diagram....11
INV-02 Manage Stock Adjustments and Restocking....11
Use Case Description....12
Use Case Diagram....13
CAP-01 Configure Starting Capital....14
Use Case Description....15
Figure 4. Configure Starting Capital use case diagram....16
VSR-01 View Sales Reports....17
Use Case Description....18
Use Case Diagram....19
Figure 5. View Sales Reports use case diagram....19
PST-02 View Order History....19
Use Case Description....20
Use Case Diagram....21

Figure 6. View Order History use case diagram 21
Entity-Relationship Diagram 22
Relationship Summary 22
Relational Schema 24
Schema Notation 24
Data Dictionary 24
Normalization Proof 28
Unnormalized Form (UNF) 28
First Normal Form (1NF) 29
Second Normal Form (2NF) 32
Third Normal Form (3NF) 35
Explicit 3NF Verification 38
Lossless-Join Proof 39
System Architecture and Technology Stack 44
GitHub Repository 46
Repository Contents 46
Deployed API 59
Android Portal API 59
Endpoints (Android App API) 59
Web Portal API 60
Endpoints (Website API) 61

SECTION 01

Project Summary

What the system is, the problem it solves, and who it is for.

Overview

Managing Sentry is a point-of-sale and inventory management system custom-built exclusively for Alekos' Corner General Merchandise. Staff process in-store cash sales through an Android POS interface, and each sale is recorded in the database while stock is updated in real time.

Problem Statement

A general merchandise store must record every sale and keep stock, expiration dates, and capital accurate. When these records are kept apart, stock counts drift from actual sales, expiring or low-stock items go unnoticed, and the owner has no single reliable view of sales performance.

Purpose and Objectives

The primary purpose of Managing Sentry is to give Alekos' Corner General Merchandise a single system that records every cash sale accurately and keeps inventory up to date in real time.

1. To process in-store cash sales through the Android POS, including total and change computation.
2.
3. To let the administrator manage products, pricing, restocking, stock adjustments, and starting capital.
4. To let the administrator review sales reports and order history.

Target Users

User	Description	What They Do in the System
Administrator	Store administrator or manager	Manages products, restocking, stock adjustments, and capital; reviews sales reports and order history
Staff (Cashier)	Store employee at the counter	Looks up products on the Android POS, enters quantities and cash received, and completes sales
Customer	Shopper buying at the counter	Presents items and pays in cash; does not use the system directly

Scope and Limitations

Scope	Android POS cash sales, real-time stock deduction, order history, product and pricing management, restocking and stock adjustments with audit records, starting capital, sales reports, and offline sales queuing (using local SQLite) with deferred synchronization.
Limitations	No barcode scanner integration (manual product entry); cash only, with no payment gateways, digital wallets, cards, or split payments; no order voids or refunds (all recorded orders are marked as COMPLETED); no multi-tenant support; no e-commerce or online ordering; and a synchronization edge case where two devices operating offline could simultaneously sell the last inventory item before syncing.

SECTION 02

System Use Cases

What the system does, described from the user's point of view.

PST-01 Process Sales Transaction

Use Case ID	PST-01
Use Case Name	Process Sales Transaction
Primary Actor	Staff (Cashier)
Secondary Actor(s)	Customer
Description	A staff member looks up items on the Android POS, enters the quantities and the cash received, and completes the sale. The server recomputes the totals from the stored product prices, saves the order, deducts the stock, and adds the sale to the store's current balance.
Trigger	Customer presents items at the counter for purchase.
Preconditions	Products exist with defined prices and available stock that has not expired. Staff is logged in to the Android POS app.
Postconditions	The order and its order items are saved, the stock of each product sold is reduced, the store's current balance is increased by the sale total, and the order is recorded as completed in the order history. If the sale is cancelled or rejected, nothing is saved and no stock is deducted.

Basic Flow

1. Customer brings the items to the counter.
2. Staff opens the Android POS interface.
3. Staff searches for the product.
4. Staff selects the product.
5. System retrieves the product details.
6. System displays the product name.
7. System displays the retail price.
8. System displays the current available stock, with a low-stock warning when the stock is at or below the alert level, and an expiry notice when the product is expiring soon or expired.
9. Staff enters the quantity to purchase.
10. System validates the requested quantity against the available stock.
11. System calculates the line subtotal.
12. System adds the selected product to the transaction.
13. Staff repeats the product selection process for additional items, if needed.
14. System calculates the total amount due.
15. Customer provides cash payment to the staff.
16. Staff enters the cash amount received into the Android POS.
17. System compares the amount tendered with the total amount due.
18. System calculates the change due.
19. System displays the calculated change.
20. System sends the sale to the server, which checks that each product has not expired and has enough stock.
21. Server recomputes the line subtotals, the total amount, and the change from the stored product prices, and ignores any prices or totals sent by the app.
22. Server saves the order and its order items, including the cost and the price at the time of the sale, which preserve historical accuracy.
23. Server deducts the purchased quantity from the product stock.
24. Server adds the total amount to the store’s current balance.
25. Server records the order as completed in the order history.
26. System displays the transaction confirmation.
27. Staff gives the calculated change to the customer.
28. Staff gives the purchased items to the customer.

Alternative Flow

14a. If the customer decides to cancel the transaction before providing payment, the staff cancels the transaction on the Android POS. The system ends the transaction and saves no order, order items, or order history record, and no stock is deducted. The POS returns to the product selection or ready state.

20a. If the Android app has no connection when the sale is confirmed, it saves the completed sale on the phone as pending. When the connection

	returns, the app sends the sale to the server and the flow continues at step 20. The server checks the sale again at that time (see 20b).
Exception Flow	8a. If the product is past its expiration date, the system shows an “Expired” notice and does not add that stock to the transaction. If the product has other stock that has not expired, the sale uses that stock instead, earliest-expiring first. Otherwise, the staff selects another product and the flow resumes at step 3. 10a. If the requested quantity is greater than the available stock, the system displays an out-of-stock alert and prevents the product from being added to the transaction (stock may never fall below zero, and checkout is blocked). The staff adjusts the quantity to the available stock or removes the item. The flow resumes at step 9. 17a. If the amount tendered is less than the total amount due, the system detects it and the staff informs the customer of the remaining balance. The customer provides additional cash, and the flow resumes at step 16. 20b. If the stock is no longer enough or the product has expired when the server checks the sale (for example, another device sold the last item first), the server rejects the sale with a “not enough stock” message and saves nothing. The staff adjusts the quantity or removes the item, and the flow resumes at step 9.
Include / Extend	None

Use Case Diagram

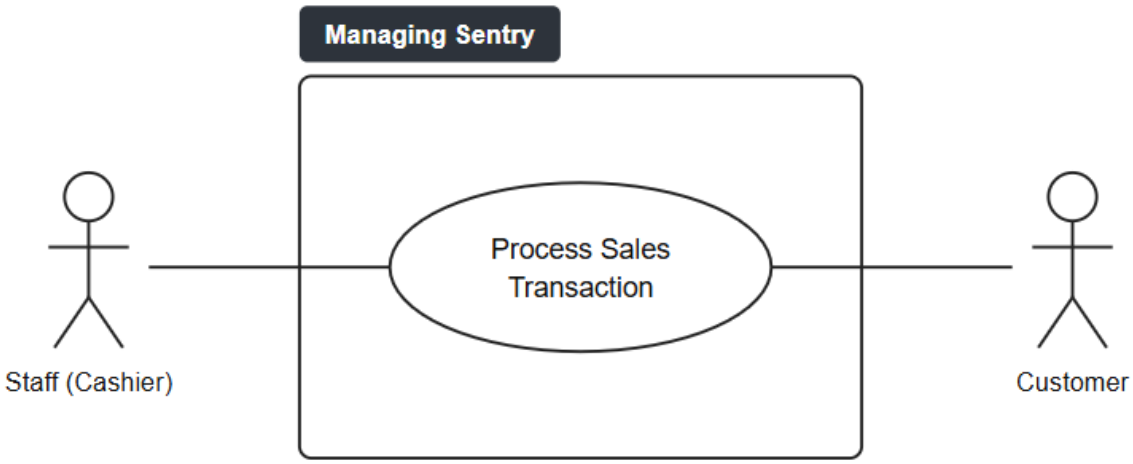


Figure 1. *Process Sales Transaction use case diagram*

INV-01 Manage Products and Inventory

Use Case Description

Use Case ID	INV-01
Use Case Name	Manage Products and Inventory
Primary Actor	Administrator
Secondary Actor(s)	None
Description	The administrator adds, edits, or deletes products and maintains each product's category, pricing, stock quantity, expiration date, and low-stock alert level. The system calculates the markup, validates and saves the information, and records each change in the stock audit log.
Trigger	The administrator needs to add, update, delete, or maintain product and inventory information.
Preconditions	The administrator is logged in to the Managing Sentry web portal with an administrator account. The product must exist when editing or deleting it.
Postconditions	The product is created, updated, or deleted, and the change is recorded in the stock audit log. The updated information is available for POS transactions and inventory monitoring.

Basic Flow	1. The administrator opens product management. 2. The system retrieves the product records. 3. The system displays the product list. 4. The administrator selects an add or edit operation. 5. The administrator enters the product name. 6. The administrator enters the product category. 7. The administrator enters the base cost. 8. The administrator enters the retail price. 9. The system calculates the markup amount as the retail price minus the base cost. 10. The administrator enters the stock quantity. 11. The administrator enters the expiration date. 12. The administrator enters the low-stock alert level. 13. The system validates the product information. 14. The system saves the product information. 15. The system records the change in the stock audit log. 16. The system displays a confirmation message.
Alternative Flow	4a. If the administrator chooses to delete a product, the system removes the product record, clears its stock, and records a “product removed” entry in the stock audit log with the product name and unit cost. The flow resumes at step 16.
Exception Flow	13a. If the system detects invalid product information, it displays an error message. The administrator corrects the information, and the system validates the corrected information. The flow resumes at step 14.
Include / Extend	None

Use Case Diagram

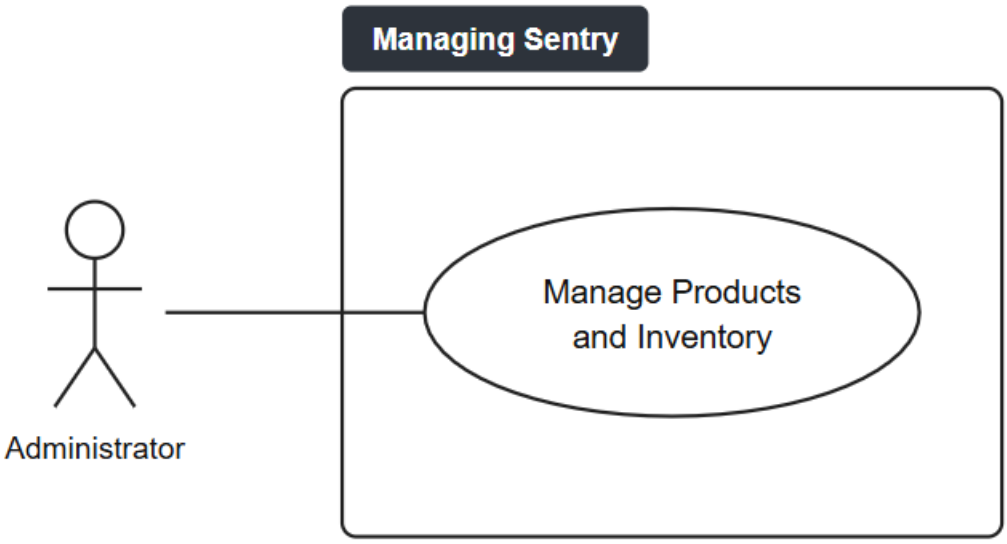


Figure 2. *Manage Products and Inventory use case diagram*

INV-02 Manage Stock Adjustments and Restocking

Use Case Description

Use Case ID	INV-02
Use Case Name	Manage Stock Adjustments and Restocking
Primary Actor	Administrator
Secondary Actor(s)	None
Description	The administrator records restocking and authorized stock removals, such as expired or damaged items, updates the product stock quantities, and maintains an audit record of each adjustment.
Trigger	The administrator needs to restock a product or remove or correct an authorized inventory quantity.
Preconditions	The administrator is logged in to the system with an administrator account. The affected product must exist.
Postconditions	The product stock quantity is updated and a stock adjustment record is created. The adjustment type, quantity, responsible user, audit notes, and transaction date are stored. Stock never falls below zero.

Basic Flow	1. The administrator opens stock adjustment management. 2. The system displays the available products. 3. The administrator selects a product. 4. The administrator selects an adjustment type (stock in or stock out). 5. The administrator enters the quantity as a positive number. The system records stock in as a positive change and stock out as a negative change. 6. The administrator enters the unit cost. 7. The administrator enters the reason or audit notes. 8. The system validates the adjustment information, including that a stock-out quantity does not exceed the available stock. 9. The system updates the product stock. When stock is removed, it is taken from the oldest or earliest-expiring batch first. 10. The system creates the stock adjustment record. 11. The system records the responsible user. 12. The system records the transaction date. 13. The system displays the adjustment confirmation.
Alternative Flow	3a. To dispose of expired stock, the administrator opens the Expiring / Expired list on the web dashboard, selects Confirm Removal for an expired item, and confirms the prompt. The system removes the expired batch from stock and records a stock-out adjustment with the reason “Confirmed disposal of expired batch”. The flow resumes at step 13.
Exception Flow	8a. If the entered quantity is invalid, the system rejects the stock adjustment. The user enters a valid quantity, and the system validates it again. The flow resumes at step 9. 8b. If a stock-out quantity is greater than the available stock, the system rejects the adjustment with a message that the quantity exceeds the available stock, and the stock remains unchanged. The user enters a valid quantity, and the system validates it again. The flow resumes at step 9.
Include / Extend	None

Use Case Diagram

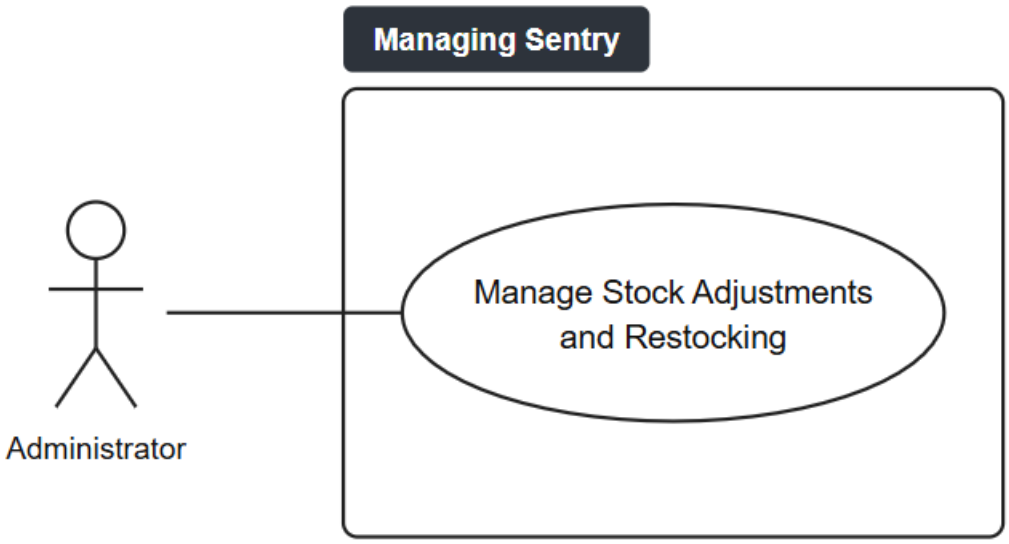


Figure 3. *Manage Stock Adjustments and Restocking use case diagram*

CAP-01 Configure Starting Capital

Use Case Description

Use Case ID	CAP-01
Use Case Name	Configure Starting Capital
Primary Actor	Administrator
Secondary Actor(s)	None
Description	The administrator establishes and maintains the store's starting capital. The store's current balance is adjusted when the starting capital changes, and it also increases with each completed sale (see PST-01).
Trigger	The administrator needs to establish or update the store's starting capital configuration.
Preconditions	The administrator is logged in to the system with an administrator account. The capital amount must be provided.
Postconditions	A new capital configuration record is saved with the starting capital, the adjusted current balance, and the update time. Earlier records are kept as a history of changes. The balance built up from sales is preserved.

Basic Flow	1. The administrator opens capital configuration. 2. The system retrieves the current configuration. 3. The system displays the current starting capital and current balance. 4. The administrator enters the starting capital. 5. The system validates the starting capital. 6. The system calculates the difference between the new and the previous starting capital. 7. The system adjusts the current balance by that difference, so the balance built up from sales is kept. On first setup, the current balance equals the starting capital. 8. The system records the update time. 9. The system saves the configuration as a new record and keeps the earlier records as history. 10. The system displays a confirmation.
Alternative Flow	None
Exception Flow	5a. If the entered capital amount is invalid, the system rejects it and the existing capital configuration remains unchanged.
Include / Extend	None

Use Case Diagram

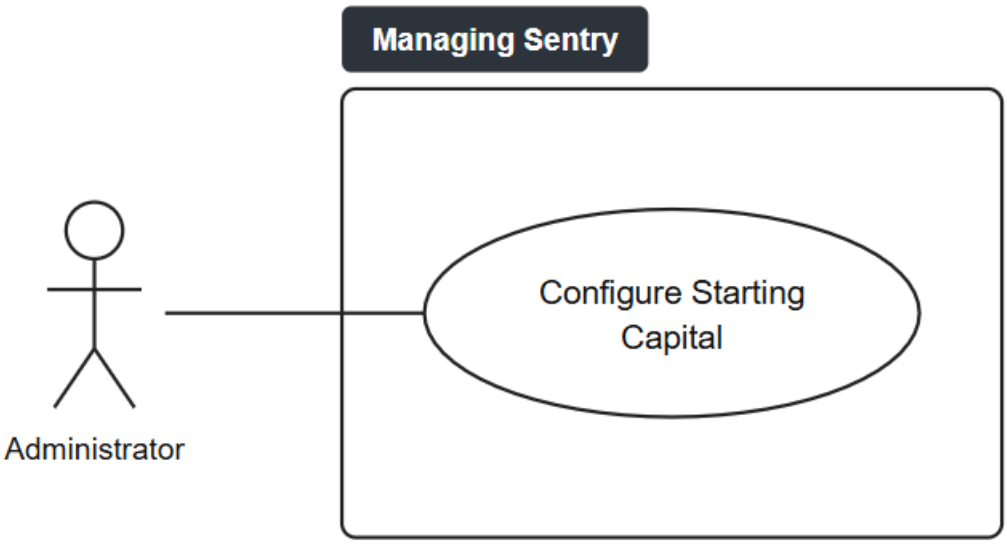


Figure 4. *Configure Starting Capital use case diagram*

VSR-01 View Sales Reports

Use Case Description

Use Case ID	VSR-01
Use Case Name	View Sales Reports
Primary Actor	Administrator
Secondary Actor(s)	None
Description	The administrator retrieves and reviews recorded sales information to monitor store transactions and support business decision-making.
Trigger	The administrator needs to review recorded sales information.
Preconditions	The administrator is logged in to the system with an administrator account, and sales records are available in the database.
Postconditions	The requested sales information is displayed to the administrator. Existing transaction records remain unchanged.
Basic Flow	1. The administrator opens the sales report module. 2. The administrator selects or enters the report criteria (such as the date range, product category, or cashier). 3. The system retrieves the matching order records. 4. The system retrieves the corresponding order-item records. 5. The system retrieves the related product information. 6. The system processes the retrieved information and calculates the sales totals. 7. The system generates the sales report. 8. The system displays the sales report. 9. The administrator reviews the report.

Alternative Flow	None
Exception Flow	3a. If the system finds no matching transactions for the selected criteria, it displays a no-records message without modifying existing data. The administrator changes the report criteria, and the flow resumes at st
Include / Extend	None

Use Case Diagram

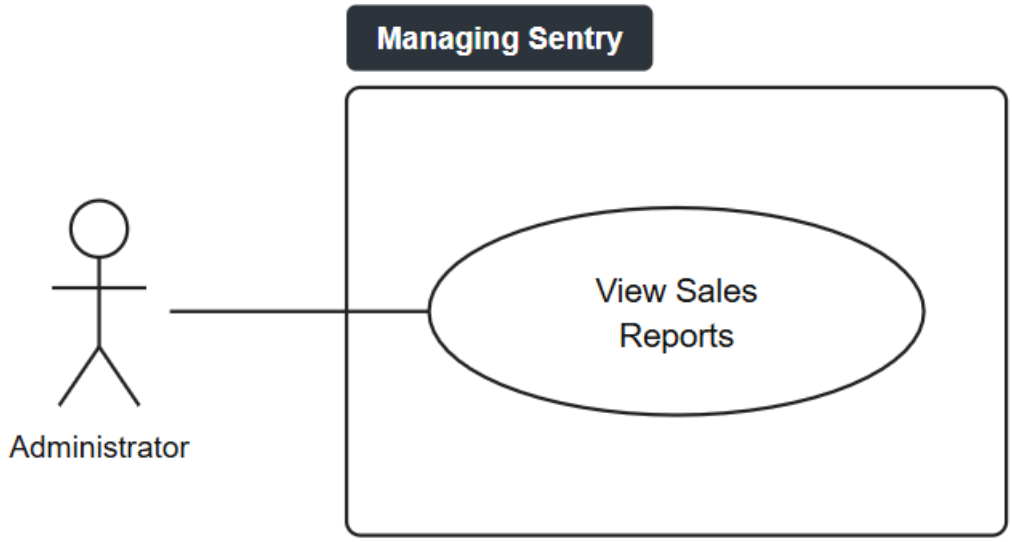


Figure 5. *View Sales Reports use case diagram*

PST-02 View Order History

Use Case Description

Use Case ID	POS-02
Use Case Name	View Order History
Primary Actor	Administrator
Secondary Actor(s)	None
Description	The administrator reviews recorded orders and their transaction statuses to monitor completed transactions and keep visibility of the order history. Orders are only viewed; voiding or refunding an order is not supported.
Trigger	The administrator needs to review a previous order or transaction status.
Preconditions	The administrator is logged in to the system. Order records exist for the requested transaction, and history records are available for completed transactions.
Postconditions	The selected order information and its transaction status are displayed. No existing transaction data is modified.

Basic Flow	1. The administrator opens order history. 2. The system retrieves the orders. 3. The system retrieves the history records. 4. The system matches the history records to the orders. 5. The system displays the order history. 6. The administrator selects an order. 7. The system retrieves the selected order. 8. The system retrieves its status history. 9. The system displays the transaction details.
Alternative Flow	None
Exception Flow	3a. If a history record is unavailable, the system does not create false history information and preserves the existing order data.
Include / Extend	None

Use Case Diagram

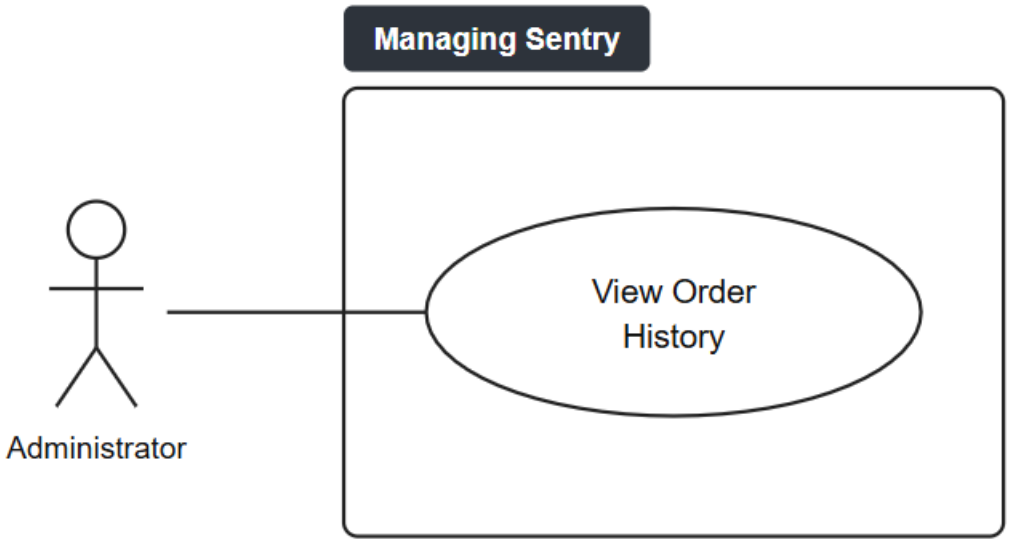


Figure 6. *View Order History use case diagram*

SECTION 03

Entity-Relationship Diagram

The entities, their attributes, and how they relate.

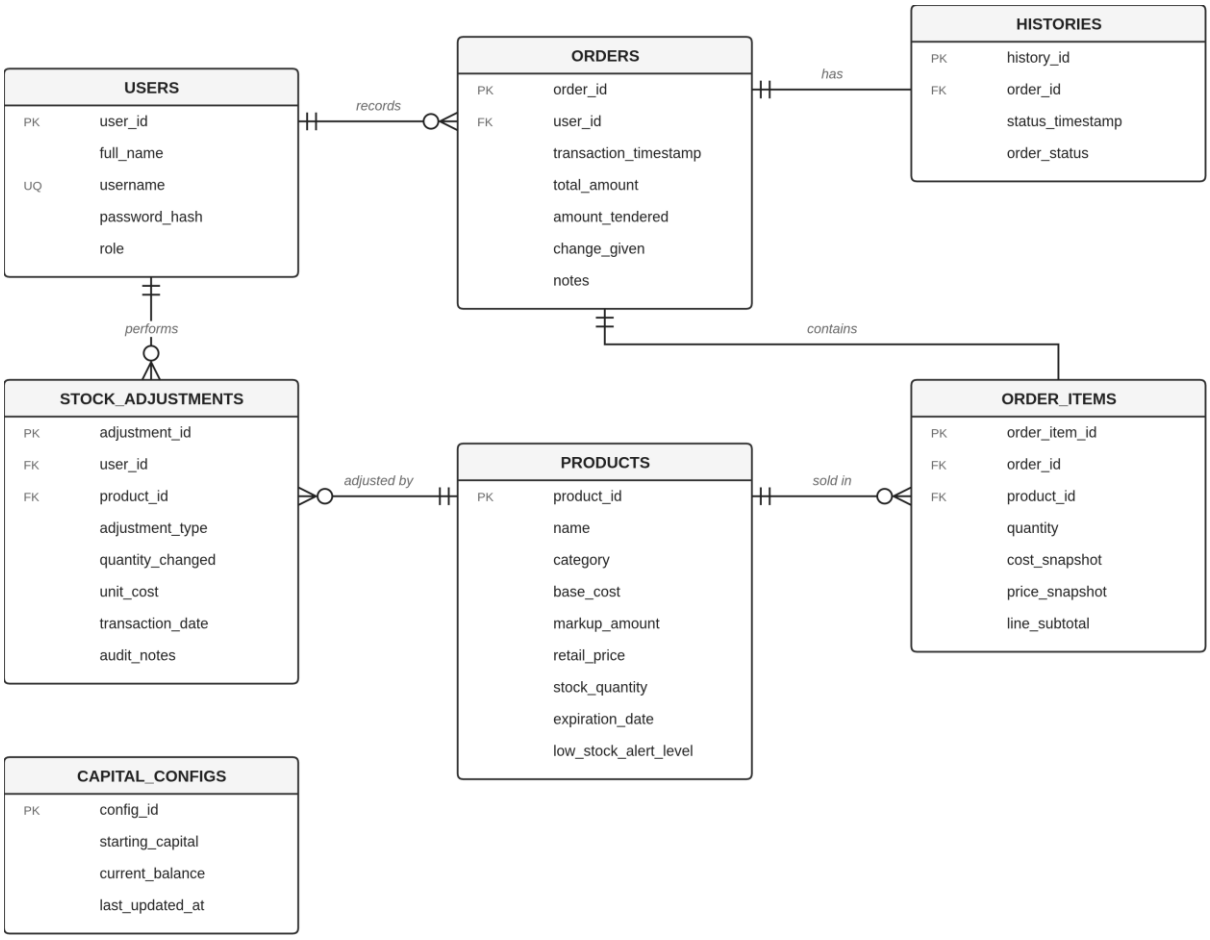


Figure 7. *Entity-Relationship Diagram of Managing Sentry*

Relationship Summary

Entity	Relationship	Related Entity	Cardinality	Business Rule
users	records	orders	1 : N	A user records zero or more orders; each order is recorded by exactly one user.
orders	contains	order_items	1 : N	An order contains one or more order items; each item belongs to exactly one order.
products	sold in	order_items	1 : N	A product appears in zero or more order items; each item refers to exactly one product. order_items resolves the many-to-many relationship between orders and products.
orders	has	histories	1 : N	An order has one or more status history records; each history record belongs to exactly one order.
users	performs	stock_adjustments	1 : N	A user performs zero or more stock adjustments; each adjustment is made by exactly one user.
products	adjusted by	stock_adjustments	1 : N	A product has zero or more stock adjustments; each adjustment refers to exactly one product.
capital_configs	—	—	None	Standalone configuration table that stores the store’s starting capital and current balance; it has no foreign key relationship in the ERD.

SECTION 04

Relational Schema

The tables, keys, and data types of the database.

Schema Notation

Legend primary_key = Primary Key foreign_key* = Foreign Key

users (user_id, full_name, username, password_hash, role)

capital_configs (config_id, starting_capital, current_balance, last_updated_at)

products (product_id, name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level)

orders (order_id, user_id*, transaction_timestamp, total_amount, amount_tendered, change_given, notes)

order_items (order_item_id, order_id*, product_id*, quantity, cost_snapshot, price_snapshot, line_subtotal)

stock_adjustments (adjustment_id, product_id*, user_id*, adjustment_type, quantity_changed, unit_cost, transaction_date, audit_notes)

histories (history_id, order_id*, status_timestamp, order_status)

Data Dictionary

users

Attribute	Data Type	Key	Constraints
user_id	SERIAL	PK	NOT NULL
full_name	VARCHAR(100)		NOT NULL
username	VARCHAR(50)		NOT NULL, UNIQUE
password_hash	VARCHAR(255)		NOT NULL
role	user_role (ENUM type: 'Admin', 'Manager', 'Staff')		NOT NULL

capital_configs

Attribute	Data Type	Key	Constraints
config_id	SERIAL	PK	NOT NULL
starting_capital	DECIMAL(12,2)		NOT NULL
current_balance	DECIMAL(12,2)		NOT NULL
last_updated_at	TIMESTAMP		NOT NULL

products

Attribute	Data Type	Key	Constraints
product_id	SERIAL	PK	NOT NULL
name	VARCHAR(100)		NOT NULL
category	VARCHAR(50)		NOT NULL
base_cost	DECIMAL(10,2)		NOT NULL
markup_amount	DECIMAL(10,2)		NOT NULL
retail_price	DECIMAL(10,2)		NOT NULL
stock_quantity	INT		NOT NULL, DEFAULT 0, CHECK (stock_quantity >= 0)
expiration_date	DATE		NULL
low_stock_alert_level	INT		NOT NULL, DEFAULT 20

orders

Attribute	Data Type	Key	Constraints
order_id	SERIAL	PK	NOT NULL
user_id	INT	FK	NOT NULL, REFERENCES users(user_id)
transaction_timestamp	TIMESTAMP		NOT NULL
total_amount	DECIMAL(10,2)		NOT NULL
amount_tendered	DECIMAL(10,2)		NOT NULL
change_given	DECIMAL(10,2)		NOT NULL
notes	TEXT		NULL

order_items

Attribute	Data Type	Key	Constraints
order_item_id	SERIAL	PK	NOT NULL
order_id	INT	FK	NOT NULL, REFERENCES orders(order_id)
product_id	INT	FK	NOT NULL, REFERENCES products(product_id) (composite UNIQUE: order_id, product_id)
quantity	INT		NOT NULL, CHECK (quantity > 0)
cost_snapshot	DECIMAL(10,2)		NOT NULL
price_snapshot	DECIMAL(10,2)		NOT NULL
line_subtotal	DECIMAL(10,2)		NOT NULL

stock_adjustments

Attribute	Data Type	Key	Constraints
adjustment_id	SERIAL	PK	NOT NULL
product_id	INT	FK	NULL, REFERENCES products(product_id)
user_id	INT	FK	NOT NULL, REFERENCES users(user_id)
adjustment_type	VARCHAR(30)		NOT NULL, CHECK (adjustment_type IN ('STOCK IN', 'STOCK OUT', 'NEW PRODUCT', 'PRODUCT EDITED', 'PRODUCT REMOVED'))
quantity_changed	INT		NOT NULL
unit_cost	DECIMAL(10,2)		NULL
transaction_date	TIMESTAMP		NOT NULL
audit_notes	TEXT		NULL

histories

Attribute	Data Type	Key	Constraints
history_id	SERIAL	PK	NOT NULL
order_id	INT	FK	NOT NULL, REFERENCES orders(order_id)
status_timestamp	TIMESTAMP		NOT NULL
order_status	VARCHAR(20)		NOT NULL, DEFAULT 'COMPLETED'

SECTION 05

Normalization Proof

From Unnormalized Form to Third Normal Form, with justification.

Unnormalized Form (UNF)

RULE

A relation is unnormalized when it contains repeating groups or attributes that hold more than one value.

SOURCE DOCUMENTS

Sales transaction (order) record, product catalog record, stock adjustment log, and order status history.

RELATION

sales_records (order_id, transaction_timestamp, total_amount, amount_tendered, change_given, notes, user_id, full_name, username, role, {products_purchased}, {stock_adjustments}, {order_history})

Braces { } enclose the repeating groups.

SAMPLE DATA

Attribute Name	Sample Value
order_id	1
transaction_timestamp	2026-03-01 10:15:00
total_amount	130.00
amount_tendered	200.00
change_given	70.00
notes	NULL
user_id	1
full_name	John Smith
username	john_admin
role	Admin

products_purchased	[1: Coffee (Category: Beverages, Qty: 2, Cost: 35.00, Markup: 15.00, Price: 50.00, Stock: 48, Exp: 2026-12-31, LowAlert: 10), 2: Bread (Category: Bakery, Qty: 1, Cost: 20.00, Markup: 10.00, Price: 30.00, Stock: 29, Exp: 2026-03-05, LowAlert: 5)]
stock_adjustments	[Adj_ID: 1, Product: 1, User: 1, Type: RESTOCK, Qty: 50, UnitCost: 35.00, Date: 2026-03-01 08:00:00, Notes: Initial stock setup]
order_history	[Hist_ID: 1, Order: 1, Time: 2026-03-01 10:15:00, Status: COMPLETED]

FUNCTIONAL DEPENDENCIES

Determinant (Left Side)	Dependent Attributes (Right Side)	Dependency Scope
order_id	transaction_timestamp, total_amount, amount_tendered, change_given, notes, user_id, full_name, username, role	Order header attributes
product_id	name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level	Product catalog attributes
(order_id, product_id)	quantity, cost_snapshot, price_snapshot, line_subtotal	Order line item attributes

JUSTIFICATION

sales_records is in UNF because a single order record holds multi-valued repeating groups: products_purchased lists two products (product_id 1 Coffee and product_id 2 Bread) in one record, and stock_adjustments and order_history carry nested logs. Store capital configuration is not part of a sales transaction, so it is not included in this record. The category attribute is included inside each product entry from the start to keep the schema consistent.

First Normal Form (1NF)

RULE

A relation is in 1NF when it has no repeating groups, every attribute holds a single atomic value, and a primary key is identified.

RELATION

sales_records (order_id, product_id, transaction_timestamp, name, category, base_cost, markup_amount, retail_price, quantity, cost_snapshot, price_snapshot, line_subtotal, total_amount, amount_tendered, change_given, notes, stock_quantity, expiration_date, low_stock_alert_level, user_id, full_name, username, role)

PRIMARY KEY

Composite primary key {order_id (PK, FK), product_id (PK, FK)}. (PK) marks a component of the primary key and (FK) a foreign key that references orders and products once the relation is decomposed.

SAMPLE DATA

The table is split into three parts for readability. order_id and product_id are repeated in each part as reference keys.

Part 1 of 3 — Order and product pricing attributes

order_id (PK, FK)	product_id (PK, FK)	transaction_timestamp	name	category	base_cost	markup_amount	retail_price	quantity	cost_snapshot	price_snapshot
1	1	2026-03-01 10:15:00	Coffee	Beverages	35.00	15.00	50.00	2	35.00	50.00
1	2	2026-03-01 10:15:00	Bread	Bakery	20.00	10.00	30.00	1	20.00	30.00

Part 2 of 3 — Transaction totals and stock attributes

order_id (PK, FK)	product_id (PK, FK)	line_subtotal	total_amount	amount_tendered	change_given	stock_quantity	expiration_date	low_stock_alert_level	user_id	notes
1	1	100.00	130.00	200.00	70.00	48	2026-12-31	10	1	NULL
1	2	30.00	130.00	200.00	70.00	29	2026-03-05	5	1	NULL

Part 3 of 3 — User attributes

order_id (PK, FK)	product_id (PK, FK)	full_name	username	role
1	1	John Smith	john_admin	Admin
1	2	John Smith	john_admin	Admin

FUNCTIONAL DEPENDENCIES

FD1 (order_id, product_id) → quantity, cost_snapshot, price_snapshot, line_subtotal — full dependency; depends on the whole composite key

FD2 product_id → name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level — partial dependency; violates 2NF

FD3 order_id → transaction_timestamp, total_amount, amount_tendered, change_given, notes, user_id, full_name, username, role — partial dependency; violates 2NF

JUSTIFICATION

The repeating group was unrolled into flat atomic rows, giving one row per product purchased in an order. order_id alone cannot be the primary key because order_id 1 appears in two rows (once for product_id 1 and once for product_id 2), so it would not identify a row. The composite key {order_id, product_id} identifies each order line (order_id 1 + product_id 1 and order_id 1 + product_id 2).

Because the key is composite, every non-key attribute must depend on both parts of it to satisfy 2NF. Product attributes depend only on product_id, and order header attributes depend only on order_id. Both are partial dependencies, and both are removed in 2NF by moving each set of attributes into its own relation (products and orders).

The store capital configuration (capital_configs: config_id, starting_capital, current_balance, last_updated_at) is a separate store-level entity. It is not part of a sales transaction and is not linked to orders, so it is kept out of the sales_records relation and stands as its own relation (capital_configs) from this point on.

Second Normal Form (2NF)

RULE

A relation is in 2NF when it is in 1NF and every non-key attribute depends on the whole primary key, with no partial dependencies.

PARTIAL DEPENDENCIES FOUND

FD2 and FD3. Product attributes depend on product_id alone, and order header attributes depend on order_id alone. Each is only part of the composite key {order_id, product_id}.

RESULTING RELATIONS

products (product_id, name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level)

orders (order_id, user_id, full_name, username, role, transaction_timestamp, total_amount, amount_tendered, change_given, notes)

order_items (order_item_id, order_id*, product_id*, quantity, cost_snapshot, price_snapshot, line_subtotal)

SAMPLE DATA

products

product_id (PK)	name	category	base_cost	markup_ amount	retail_price	stock_quantity	expiration_ date	low_stock_ alert_level
1	Coffee	Beverages	35.00	15.00	50.00	48	2026-12-31	10
2	Bread	Bakery	20.00	10.00	30.00	29	2026-03-05	5

orders (part 1 of 2)

order_id (PK)	user_id	full_name	username
1	1	John Smith	john_admin

orders (part 2 of 2)

order_id (PK)	role	transaction_ timestamp	total_amount	amount_ tendered	change_ given	notes
1	Admin	2026-03-01 10:15:00	130.00	200.00	70.00	NULL

order_items

order_ item_id (PK)	order_id (FK)	product_id (FK)	quantity	cost_snapshot	price_snapshot	line_subtotal

1	1	1	2	35.00	50.00	100.00
2	1	2	1	20.00	30.00	30.00

DEPENDENCIES REMAINING FOR 3NF

Location	Dependency Identified	Classification	Action Required for 3NF
orders table	user_id → full_name, username, role	Transitive dependency (order_id → user_id → full_name)	Extract to users table

JUSTIFICATION

Attributes that depend only on product_id were moved to products, and attributes that depend only on order_id were moved to orders. order_items keeps the attributes that depend on the order line, and uses a surrogate key order_item_id with order_id and product_id as foreign keys. No partial dependency remains.

capital_configs (config_id, starting_capital, current_balance, last_updated_at) is a separate entity from the order record. It describes the store's cash configuration, which exists independently of any single order, and it is not linked to orders through a foreign key. Because orders holds no config_id, order_id → config_id is not a real dependency, so the capital attributes are not a transitive dependency of orders. They were therefore moved out of the order data in 1NF and remain a standalone store-level table (orders does not reference it).

Third Normal Form (3NF)

RULE

A relation is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute, with no transitive dependencies.

TRANSITIVE DEPENDENCIES FOUND

order_id → user_id → full_name, username, role

FINAL RELATIONS

users (user_id, full_name, username, password_hash, role)

capital_configs (config_id, starting_capital, current_balance, last_updated_at)

products (product_id, name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level)

orders (order_id, user_id*, transaction_timestamp, total_amount, amount_tendered, change_given, notes)

order_items (order_item_id, order_id*, product_id*, quantity, cost_snapshot, price_snapshot, line_subtotal)

stock_adjustments (adjustment_id, product_id*, user_id*, adjustment_type, quantity_changed, unit_cost, transaction_date, audit_notes)

histories (history_id, order_id*, status_timestamp, order_status)

SAMPLE DATA

users

user_id (PK)	full_name	username	password_hash	role
1	John Smith	john_admin	2b12elmigU...	Admin
2	Jack Davis	jack_cashier	2b12kP9qL...	Staff

capital_configs

config_id (PK)	starting_capital	current_balance	last_updated_at
1	10000.00	10130.00	2026-03-01 08:00:00

products

product_id (PK)	name	category	base_ cost	markup_ amount	retail_ price	stock_ quantity	expiration_date	low_ stock_ alert_ level

1	Coffee	Beverages	35.00	15.00	50.00	48	2026-12-31	10
2	Bread	Bakery	20.00	10.00	30.00	29	2026-03-05	5

orders

order_id (PK)	user_id (FK)	transaction_ timestamp	total_amount	amount_tendered	change_given	notes
1	1	2026-03-01 10:15:00	130.00	200.00	70.00	NULL

order_items

order_ item_id (PK)	order_id (FK)	product_id (FK)	quantity	cost_snapshot	price_snapshot	line_subtotal
1	1	1	2	35.00	50.00	100.00
2	1	2	1	20.00	30.00	30.00

stock_adjustments

adjustment_id (PK)	product_id (FK)	user_id (FK)	adjustment_type	quantity_ changed	unit_cost	transaction_date	audit_notes
1	1	1	RESTOCK	50	35.00	2026-03-01 08:00:00	Initial stock setup

histories

history_id (PK)	order_id (FK)	status_timestamp	order_status
1	1	2026-03-01 10:15:00	COMPLETED

JUSTIFICATION

Transitive and non-key dependencies were removed so that every non-key attribute depends only on the primary key. User details were moved to users, leaving user_id in orders as a foreign key. capital_configs stays a standalone store-level table, kept as its own relation since 1NF and not linked to orders. The users relation also stores password_hash, an authentication attribute of the system that does not appear on the source documents. Every non-key attribute now depends on the key, the whole key, and nothing but the key. These seven relations are in 3NF.

Explicit 3NF Verification

Each final relation is verified below, confirming that every non-key attribute depends only on the primary key.

Relation Name	Primary Key	Direct Functional Dependency Statement	Transitive / Partial Dependencies Present?	3NF Status
users	user_id	user_id → full_name, username, password_hash, role	None	Satisfied
capital_configs	config_id	config_id → starting_capital, current_balance, last_updated_at	None	Satisfied
products	product_id	product_id → name, category, base_cost, markup_amount, retail_price, stock_quantity, expiration_date, low_stock_alert_level	None	Satisfied
orders	order_id	order_id → user_id, transaction_timestamp, total_amount, amount_tendered, change_given, notes	None	Satisfied
order_items	order_item_id	order_item_id → order_id, product_id, quantity, cost_snapshot, price_snapshot, line_subtotal	None	Satisfied
stock_adjustments	adjustment_id	adjustment_id → product_id, user_id, adjustment_type, quantity_changed, unit_cost, transaction_date, audit_notes	None	Satisfied
histories	history_id	history_id → order_id, status_timestamp, order_status	None	Satisfied

Lossless-Join Proof

To prove that the decomposition from 1NF into 3NF is lossless, natural joins are executed across orders, order_items, and products for order_id 1.

RECONSTRUCTION CHAIN

order_id 1 → orders → order_items → products. Joining orders with order_items on order_id, then joining the result with products on product_id, must recreate the complete original order information with no missing or duplicated rows.

The complete UNF record is losslessly reconstructed through the dependency paths orders → order_items → products, orders → users, histories joined to orders on order_id, and stock_adjustments joined to products on product_id and to users on user_id. capital_configs is a standalone store-level table that is not part of the UNF record, so it takes no part in the join, stock_adjustments and histories need no separate 1NF or 2NF step of their own, as explained below. Each decomposition keeps the determinant as a key or foreign key, so no original information is lost and no spurious tuples are generated.

Why stock_adjustments and histories need no separate 1NF or 2NF steps: in the UNF they are the repeating groups {stock_adjustments} and {order_history}, and they appear as their own relations in 3NF. Each step is shown here.

1NF: each log entry already holds only single-valued attributes, so unrolling the group gives one atomic row per adjustment (key adjustment_id) and one atomic row per status record (key history_id), each with its own primary key.

2NF: both relations have a single-attribute primary key, so a partial dependency is not possible and 2NF holds with no change.

3NF: every non-key attribute depends directly on adjustment_id or history_id, and no non-key attribute determines another, so 3NF holds. The only change needed was to separate them into their own relations, with product_id and user_id as foreign keys in stock_adjustments and order_id as the foreign key in histories.

A surrogate order_item_id is introduced in 2NF and kept in 3NF as the primary key for each physical order line. It simplifies foreign key references from other tables, such as future returns, discounts, or line-level adjustments. The composite key {order_id, product_id} remains a valid candidate key because it still identifies a product within an order, and it can be enforced with a UNIQUE constraint.

RECONSTRUCTED ORDER DETAIL FOR ORDER_ID 1

- product_id 1 Coffee (Beverages) — Qty 2 × Price ₱50.00 = Subtotal ₱100.00
- product_id 2 Bread (Bakery) — Qty 1 × Price ₱30.00 = Subtotal ₱30.00

- Order Total = ₱100.00 + ₱30.00 = ₱130.00, matching orders.total_amount for order_id 1 exactly.

NATURAL JOIN STEP 1: orders ⋈ order_items ON order_id

Part 1 of 2 — order attributes

order_id	user_id	transaction_timestamp	total_amount	amount_ tendered	change_given	notes
1	1	2026-03-01 10:15:00	130.00	200.00	70.00	NULL
1	1	2026-03-01 10:15:00	130.00	200.00	70.00	NULL

Part 2 of 2 — order item attributes

order_id	order_ item_id	product_id	quantity	cost_snapshot	price_snapshot	line_subtotal
1	1	1	2	35.00	50.00	100.00
1	2	2	1	20.00	30.00	30.00

NATURAL JOIN STEP 2: Temp1 ⋈ products ON product_id

order_id	product_id	transaction_ timestamp	name	category	quantity	retail_ price	line_ subtotal	total_amount	user_id
1	1	2026-03-01 10:15:00	Coffee	Beverages	2	50.00	100.00	130.00	1
1	2	2026-03-01 10:15:00	Bread	Bakery	1	30.00	30.00	130.00	1

FORMAL LOSSLESS PROOF VERIFICATION

Evaluation Parameter	Mathematical Condition	Result / Verification
Intersection attribute	R1 ∩ R2 = {order_id}	Common joining key across orders and order_items
Key functional dependency	R1 ∩ R2 → R1	order_id is a superkey of the orders relation
Tuple loss verification	Count(Reconstructed) = 2	Exactly 2 rows returned, matching the original 1NF record count

Spurious tuple check	Spurious Rows = 0	No phantom or extraneous rows generated
Data integrity outcome	Line 1: 2 × 50 = 100; Line 2: 1 × 30 = 30; Total: 130.00	Lossless join proven

SECTION 06

System Architecture and Technology Stack

How the parts of the system connect, and the tools used to build it.

Architecture Diagram

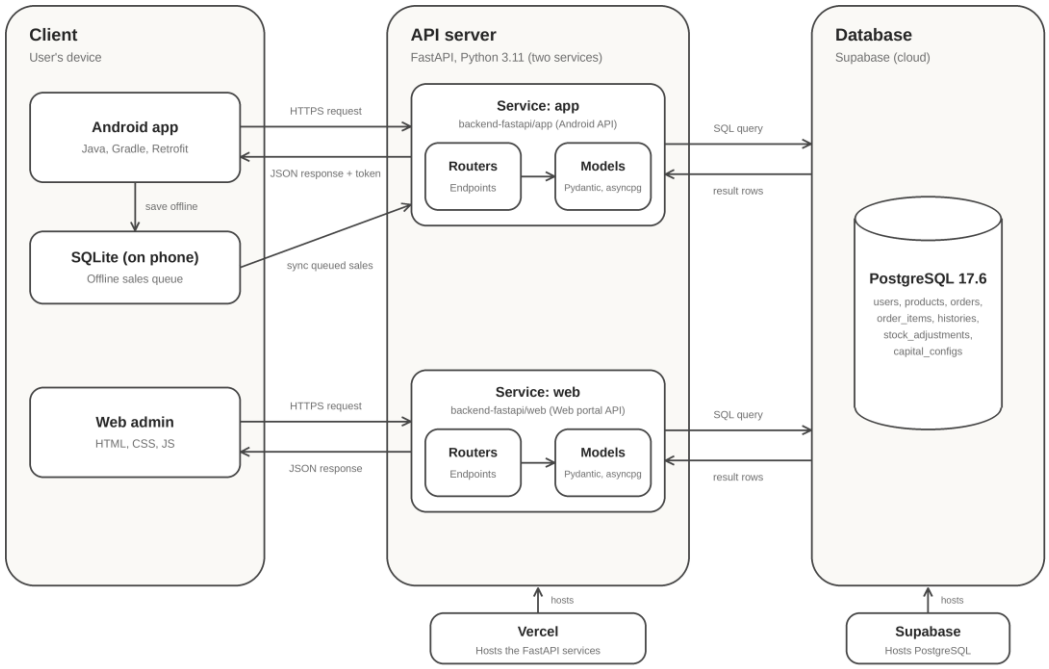


Figure 8. *System architecture of Managing Sentry*

Architecture Description

Managing Sentry follows a three-tier client-server architecture. A client, either the Android POS app or the web admin portal, sends an HTTPS request to the REST API. The API is a FastAPI backend (Python) hosted on Vercel. An earlier PHP prototype remains in the repository but is no longer used. It will be moved to an archive branch or removed in a later cleanup, not as part of this submission. Several FastAPI routes still accept the old .php paths (for example, /make_sale.php) and are kept for older versions of the Android app. The routers receive the request, validate it with Pydantic models, and apply the store's rules, such as computing sale totals and change and updating stock after a checkout. The data layer uses asyncpg to read and write records in the PostgreSQL database hosted on Supabase, and the API returns the result to the client as JSON.

Technology Stack

Layer	Technology	Role in the System
Client	Android app (Java 11, Android SDK 37, Gradle 9.5.0 (AGP Plugin 9.3.1)), Retrofit 2.11.0; web admin portal (HTML5, CSS3, JavaScript)	Sends requests to the API and displays the responses, such as the cart, receipts, inventory, history, and the dashboard
Local storage	SQLite	Stores transactions offline on the Android app and syncs them when the connection returns
Backend	Python 3.11, FastAPI 0.115.0, Uvicorn 0.32.0	Defines the endpoints and applies the business rules, such as sale totals, change, and stock updates
Data access	asyncpg 0.30.0, Pydantic 2.10.0	Runs SQL queries on the database and validates request and response data
Database	PostgreSQL [17.6] (Supabase)	Stores products, stock, sales transactions, stock adjustments, order history, and user accounts
Hosting	Vercel (API), Supabase (database)	Runs the live FastAPI API and the database
Version control	Git, GitHub	Tracks changes to the project
Tools	Android Studio, VS Code, pgAdmin, Figma	Used for coding, database management, and UI design

SECTION 07

GitHub Repository

Where the source code and database scripts live.

Repository URL	https://github.com/serious-capybara/managing-sentry
Visibility	Public

Repository Contents

Item	Location	Description

README	/README.md	System overview, tech stack, architecture, and step-by-step setup and run instructions

Contributing guide	/CONTRIBUTING.md	Branching strategy, commit and push rules, and pull request guidelines

Android app source code	/app	Android POS app (Java, Gradle) with the dashboard, products, and history screens, plus phone and tablet layouts

Web portal source code	/web	Web admin portal (HTML, CSS, JavaScript) that calls the API

Legacy PHP backend	/backend	Legacy PHP 8.2 prototype (endpoint handlers, database configuration, and core utilities) that has been replaced by the FastAPI backend and is no longer used

FastAPI backend	/backend-fastapi	Python FastAPI API with routers, Pydantic models, database connection, and security configuration

Database scripts	/db	Supabase schema and seed data (dump-postgres-202610150650.sql), which creates the tables and loads the existing records

Environment sample	/backend-fastapi/web/.env.example	Required environment variables with placeholder values (web service only; the app service has no .env.example)

Dependencies	/backend-fastapi/requirements.txt	Python packages needed to run the FastAPI backend (the Android app uses Gradle)

Git ignore	/.gitignore	Files and folders excluded from version control

SECTION 08

Deployed API

The live API and the endpoints it exposes.

Android Portal API

Framework	FastAPI (Python 3.11)
Base URL	https://managing-sentry-app.vercel.app
API Documentation	https://managing-sentry-app.vercel.app/docs (Swagger UI) and https://managing-sentry-app.vercel.app/redoc (ReDoc)
Hosting Platform	Vercel
Database	PostgreSQL (Supabase)
Test Credentials	Test admin username: admin Test admin password: matas1510#admin Test staff username: epsilon Test staff password: epsilon#7 Docsusername*:* alekos Docspassword: merchandise#1015520

Endpoints (Android App API)

Method	Endpoint	Description	Use Case
GET	/	Root route that confirms the API is running	NONE
POST	/login	Authenticates a user by username and password, and returns the user details, role, and an access token that must be sent with every other request	ALL
GET	/products	List all products	PST-01, INV-01
GET	/categories	List the product categories	INV-01
POST	/sales	Create an order and deduct the stock	PST-01
GET	/history	List orders and their status	PST-02
GET	/health	Health check that confirms the server is working	NONE

Web Portal API

Framework	FastAPI (Python 3.11)
Base URL	https://managing-sentry-web.vercel.app
API Documentation	https://managing-sentry-web.vercel.app/docs (Swagger UI) and https://managing-sentry-web.vercel.app/redoc (ReDoc)
Hosting Platform	Vercel
Database	PostgreSQL, Supabase
Test Credentials	Test admin username: admin Test admin password: matas1510#admin Test staff username: epsilon Test staff password: epsilon#7 Docs username*:* alekos Docs password: merchandise#1015520

Endpoints (Website API)

Method	Endpoint	Description	Use Case
GET	/	Root route that confirms the API is running	NONE
POST	/login	Authenticates a user by username and password, and returns the user details, role, and an access token that must be sent with every other request	ALL
GET	/products	Lists all inventory products with stock levels, costs, prices, and sales totals	POS-01, INV-01, INV-02, VSR-01
POST	/products	Performs product operations: creates a new product, edits details, adjusts stock levels, or deletes a product.	INV-01, INV-02
GET	/capital	Retrieves the starting capital, current balance, and last updated timestamp.	CAP-01, VSR-01
POST	/capital	Sets or updates the starting capital and current balance.	CAP-01
POST	/sales	Creates a new sale order, records item line details, deducts product stock, logs history, and updates capital balance.	PST-01
GET	/history	Lists all historical sales orders and stock movement adjustments sorted by timestamp.	PST-02, VSR-01, INV-02
GET	/sync_status	Generates and returns an MD5 database version hash to check if cached client data is up to date.	**NONE**
GET	/maintenance	Performs database diagnostic tests, inspects table schemas, or applies schema column updates.	NONE
POST	/reset	Resets all sales records, order items, histories, and stock adjustments, zeroes stock levels, and resets capital.	NONE
GET	/health	Health check endpoint that confirms the server is active and operational.	NONE